

BLUESTOCK MUTUAL FUND ANALYTICS PLATFORM

Analyst Guide

N100 Financial Intelligence Dashboard

Submitted in Partial Fulfillment of the Requirements for the
Data Analyst Internship Program

Prepared By

Anushika Kapoor

Computer Science Engineering

Date of Submission: July 2026

Project Manager

Yash Kale

Bluestock Fintech

INDEX

1. Chapter 1: Introduction.....	3
2. Chapter 2: System Architecture & Workflow.....	8
3. Chapter 3: Installation & System Setup.....	13
4. Chapter 4: Dashboard Modules.....	18
5. Chapter 5: API, Testing, Report Generation & Troubleshooting.....	25
6. Chapter 6: Conclusion & Future Enhancements.....	31

CHAPTER 1: INTRODUCTION

1.1 Introduction

The **N100 Financial Intelligence Dashboard** is a comprehensive financial analytics platform developed to simplify the analysis of companies listed in the **Nifty 100 Index**. Financial analysis typically involves gathering data from multiple financial statements, calculating key performance indicators, comparing companies within industries, and interpreting complex financial information. Performing these tasks manually can be time-consuming, repetitive, and susceptible to human error. This project addresses these challenges by integrating an automated **Extract, Transform, and Load (ETL)** pipeline, a centralized **SQLite database**, **FastAPI-based backend services**, and an interactive **Streamlit dashboard** into a unified system. The platform automates financial data processing and provides users with meaningful insights through interactive visualizations, company comparisons, AI-generated summaries, and downloadable reports.

The dashboard enables analysts and investors to evaluate company performance using financial ratios, valuation metrics, profitability indicators, cash flow analysis, peer comparison, and sector-level insights. By presenting financial information in an organized and interactive manner, the system supports faster and more informed investment decisions.

1.2 Purpose of the Analyst Guide

This Analyst Guide serves as the official user manual for the N100 Financial Intelligence Dashboard. It explains the architecture, functionality, installation process, dashboard modules, API services, reporting capabilities, and testing procedures of the application.

The guide is intended to help users understand how the system operates and how each module contributes to financial analysis. It also provides sufficient information for developers, project evaluators, and future maintainers to understand the design and implementation of the application. The document aims to ensure consistent usage of the dashboard while reducing the learning curve for first-time users.

1.3 Project Objectives

The primary objective of the project is to develop an integrated financial analytics platform capable of transforming raw financial datasets into meaningful business insights through automation and visualization.

The specific objectives include:

- To automate the extraction, cleaning, validation, and storage of financial datasets.
- To calculate important financial ratios and performance indicators automatically.

- To provide interactive dashboards for financial analysis.
- To enable comparison of companies within the same industry.
- To generate AI-assisted financial summaries and recommendations.
- To support portfolio analysis and valuation assessment.
- To provide downloadable analytical reports in PDF and CSV formats.
- To expose financial data through RESTful APIs using FastAPI.
- To improve the efficiency and accuracy of financial decision-making.

1.4 Scope of the Project

The scope of the N100 Financial Intelligence Dashboard covers the complete financial analytics workflow, beginning with raw financial statement data and ending with interactive dashboards and analytical reports.

The application processes financial information for companies included in the Nifty 100 index. The datasets include:

- Company information
- Profit and Loss Statements
- Balance Sheets
- Cash Flow Statements
- Financial Ratios
- Market Capitalization
- Valuation Metrics
- Business Analysis
- AI-generated Recommendations

The dashboard provides multiple analytical modules, including:

- Industry Overview
- Company Profile
- Stock Screener
- Sector Analysis
- Peer Comparison
- Valuation Analysis
- Capital Allocation
- Portfolio Summary
- AI Insights
- Reports

Although the current implementation is designed for offline analytical use with SQLite, the modular architecture allows future integration with cloud databases and real-time financial data sources.

1.5 Target Users

The application is designed for a wide range of users involved in financial analysis and investment research.

Financial Analysts

Financial analysts can evaluate company performance, study historical trends, compare competitors, and generate professional reports using the dashboard.

Investment Advisors

Investment professionals can analyze company fundamentals, compare valuation metrics, and make informed recommendations based on financial indicators.

Equity Researchers

Researchers can utilize the application to perform sector-wise analysis, peer comparison, and profitability evaluation while preparing equity research reports.

Students

Students pursuing finance, business analytics, commerce, or data science can use the dashboard as a practical learning tool for understanding financial statements and financial ratio analysis.

Individual Investors

Retail investors can analyze companies, compare investment opportunities, and understand company fundamentals before making investment decisions.

1.6 Key Features

The N100 Financial Intelligence Dashboard integrates multiple analytical features within a single platform.

Some of the major features include:

- Automated ETL pipeline for financial data processing.
- Centralized SQLite database for structured storage.
- Interactive Streamlit dashboard.
- FastAPI backend for RESTful API services.
- Financial ratio calculations including ROE, ROCE, EPS, Current Ratio, Debt-to-Equity Ratio, and Net Profit Margin.
- Company-wise financial analysis.
- Industry and sector performance comparison.

- Peer comparison using radar charts and financial metrics.
- Valuation analysis based on financial ratios.
- Capital allocation and cash flow analysis.
- Portfolio summary generation.
- AI-generated company summaries and recommendations.
- Downloadable PDF reports and CSV exports.
- Interactive filtering and visualization.

1.7 Technology Stack

The application has been developed using modern open-source technologies that support efficient financial data processing and visualization.

Layer	Technology
Programming Language	Python 3.11
Frontend	Streamlit
Backend	FastAPI
Database	SQLite
Data Processing	Pandas, NumPy
Visualization	Plotly, Matplotlib
API Documentation	Swagger (OpenAPI)
Report Generation	ReportLab
Testing	Pytest
Version Control	Git & GitHub

Each technology has been selected based on its performance, ease of integration, scalability, and community support.

1.8 Benefits of the System

The dashboard offers several advantages over traditional spreadsheet-based financial analysis.

- Eliminates repetitive manual calculations.
- Reduces the possibility of calculation errors.
- Provides centralized access to financial information.
- Enables quick comparison of companies and sectors.
- Supports faster investment analysis.
- Generates professional reports automatically.
- Improves decision-making through interactive visualizations.
- Offers scalable architecture for future enhancements.

1.9 Organization of the Analyst Guide

This Analyst Guide is organized into eight chapters, each covering a specific aspect of the project.

- **Chapter 1** introduces the project, objectives, scope, target users, and technology stack.
- **Chapter 2** explains the overall system architecture, workflow, and major system components.
- **Chapter 3** describes the installation procedure, software requirements, and execution steps.
- **Chapter 4** provides a detailed walkthrough of each dashboard module and its analytical features.
- **Chapter 5** documents the RESTful APIs and explains their functionality.
- **Chapter 6** discusses report generation and data export features.
- **Chapter 7** presents the testing methodology and troubleshooting guidelines.
- **Chapter 8** concludes the guide and outlines possible future enhancements.

1.10 Chapter Summary

This chapter introduced the N100 Financial Intelligence Dashboard and described its purpose, objectives, scope, intended users, key features, and underlying technology stack. The project has been developed to automate financial data processing and provide a centralized platform for analyzing Nifty 100 companies using interactive dashboards, financial ratios, AI-assisted insights, and downloadable reports. The following chapter explains the system architecture and illustrates how the various components interact to deliver financial intelligence.

CHAPTER 2: SYSTEM OVERVIEW AND ARCHITECTURE

2.1 Introduction

The N100 Financial Intelligence Dashboard is developed as a modular financial analytics platform that combines data engineering, database management, backend API services, and interactive visualization into a single integrated solution. The application is designed to automate the complete workflow of financial analysis, beginning with raw financial datasets and ending with dynamic dashboards, AI-generated insights, and downloadable reports.

The system eliminates the need for manual financial calculations by automatically processing company financial statements, computing key performance indicators, and presenting them through interactive charts and analytical dashboards. The modular architecture ensures that each component of the application performs a dedicated responsibility while maintaining seamless communication with the remaining modules.

2.2 System Workflow

The overall workflow of the application follows a structured pipeline consisting of five major stages.

Stage 1: Data Collection

The first stage involves collecting financial datasets from various Excel files. These datasets contain company information, profit and loss statements, balance sheets, cash flow statements, financial ratios, valuation metrics, market capitalization data, and qualitative business analysis.

Since the datasets originate from multiple sources, inconsistencies such as missing values, duplicate records, inconsistent column names, and varying date formats are expected.

Stage 2: ETL Pipeline

The Extract, Transform, and Load (ETL) pipeline is responsible for preparing the raw datasets before they are used for analytics.

The ETL module performs the following operations:

- Extracts data from raw Excel files.
- Cleans invalid or duplicate records.
- Standardizes column names and financial values.
- Normalizes company identifiers.
- Converts inconsistent date and numeric formats.
- Validates mandatory fields.
- Loads the cleaned datasets into the SQLite database.

This automated pipeline ensures consistency and accuracy across all financial datasets.

Stage 3: Database Layer

After processing, the cleaned data is stored in an SQLite relational database.

The database acts as the central repository for all financial information and provides structured storage for company profiles, financial statements, ratio calculations, valuation metrics, peer comparison data, and analytical reports.

SQLite was selected because it is lightweight, portable, requires no server installation, and offers efficient performance for analytical applications.

The application also utilizes indexing techniques to improve query execution speed, particularly for large financial datasets.

Stage 4: Backend Services

The backend is developed using FastAPI.

FastAPI exposes multiple RESTful API endpoints that retrieve financial information directly from the SQLite database. These APIs serve as the communication layer between the database and the Streamlit dashboard.

Major API modules include:

- Company Information API
- Industry Overview API
- Stock Screener API
- Sector Analysis API
- Peer Comparison API
- Valuation API
- Portfolio Analysis API
- Report Generation API

The backend also provides automatic API documentation through Swagger UI.

Stage 5: Dashboard and Reporting

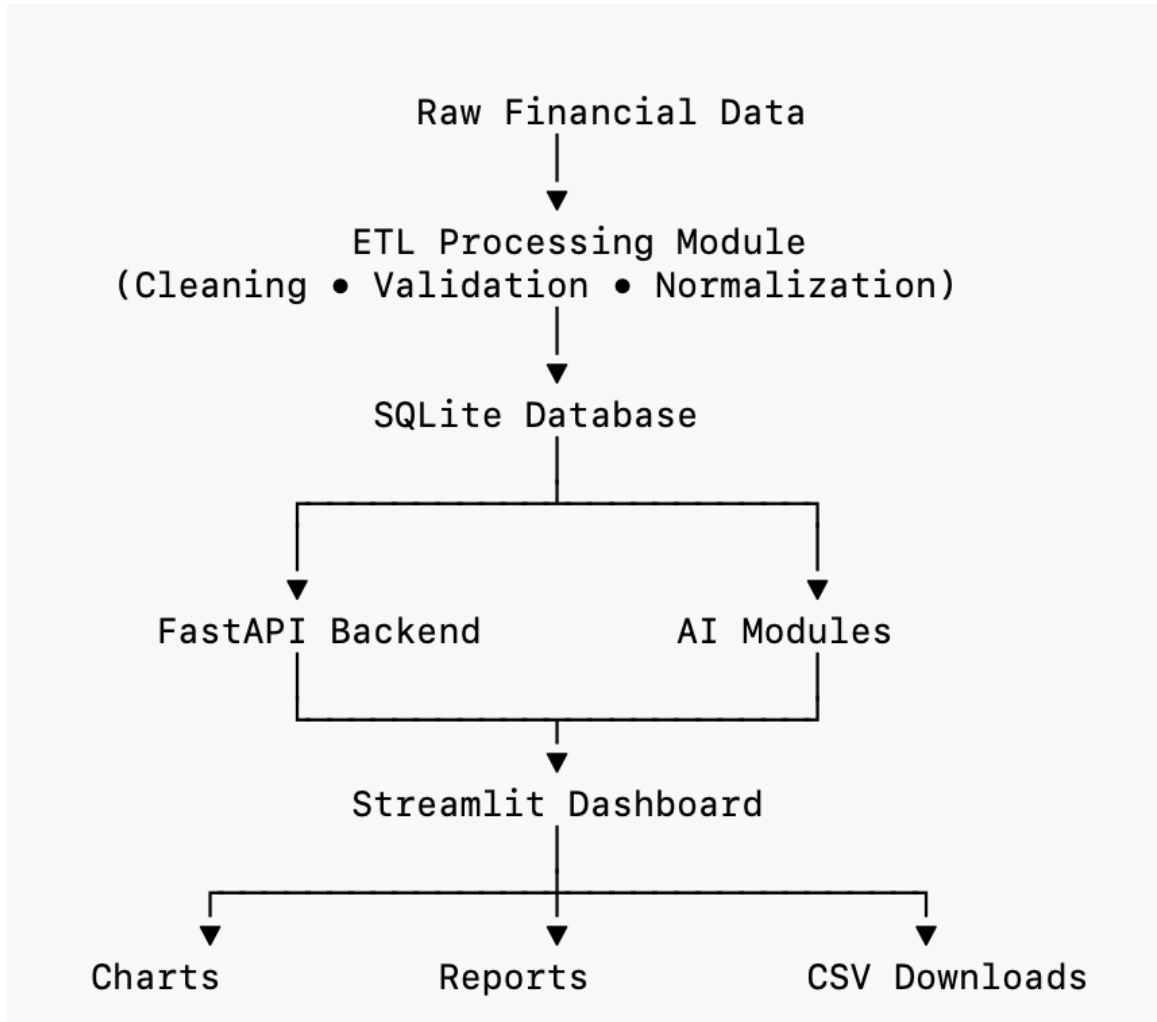
The frontend of the application is built using Streamlit.

The dashboard allows users to interactively explore financial information using filters, charts, tables, and analytical visualizations. Users can compare companies, analyze financial ratios, review sector performance, generate reports, and download CSV or PDF files.

The reporting module creates professional analytical documents that summarize company performance and portfolio statistics.

2.3 Overall System Architecture

The architecture of the application follows a layered design that separates data processing, storage, business logic, and visualization.



2.4 Major System Components

Data Processing Module

The data processing module forms the foundation of the application. It converts raw financial datasets into structured information suitable for analysis. Data validation rules ensure consistency before loading into the database.

Database Module

The SQLite database stores all processed datasets and provides efficient retrieval of financial information. Multiple tables are linked using company identifiers, enabling seamless joins across financial statements, ratios, and valuation data.

Analytics Module

The analytics module computes various financial metrics, including:

- Return on Equity (ROE)
- Return on Capital Employed (ROCE)
- Earnings Per Share (EPS)
- Debt-to-Equity Ratio
- Current Ratio
- Profit Margins
- Revenue Growth
- Cash Flow Indicators

These metrics are used throughout the dashboard to support financial analysis.

AI Insights Module

The AI module analyzes financial ratios and company performance to generate concise business summaries, highlight strengths and weaknesses, and provide recommendation statements. This feature assists users in quickly understanding a company's financial position.

Visualization Module

The visualization layer presents financial information through interactive charts and graphs using Plotly and Streamlit.

Common visualizations include:

- Line Charts
- Bar Charts
- Pie Charts
- Scatter Plots
- Radar Charts
- KPI Cards
- Trend Analysis Charts

Interactive filtering allows users to compare companies across multiple financial indicators.

Reporting Module

The reporting module generates downloadable documents summarizing financial performance. Reports include graphical analysis, ratio summaries, valuation metrics, and company-specific insights. Users can export reports in PDF and CSV formats for further analysis or sharing.

2.5 Data Flow

The data flow within the application follows a sequential process:

1. Raw financial datasets are imported into the ETL pipeline.
2. Data cleaning and normalization are performed.
3. Cleaned data is stored in the SQLite database.
4. FastAPI retrieves requested information through SQL queries.
5. Streamlit displays the processed information as interactive dashboards.
6. Users generate reports or download analytical data as required.

This structured workflow ensures accurate, consistent, and efficient delivery of financial insights.

2.6 Advantages of the Architecture

The modular architecture offers several benefits:

- Clear separation of responsibilities between components.
- Easy maintenance and future enhancements.
- Efficient handling of financial datasets.
- Faster database queries through indexing.
- Interactive user experience with real-time filtering.
- Reusable REST APIs for integration with external applications.
- Scalable design that supports the addition of new analytical modules.

2.7 Chapter Summary

This chapter described the overall architecture and workflow of the N100 Financial Intelligence Dashboard. It explained how raw financial data is transformed into meaningful analytical insights through the ETL pipeline, SQLite database, FastAPI backend, AI modules, and Streamlit dashboard. The layered architecture improves maintainability, scalability, and performance while providing users with an efficient platform for financial analysis. The next chapter presents the installation and configuration process required to deploy and execute the application.

CHAPTER 3: INSTALLATION AND SYSTEM SETUP

3.1 Introduction

This chapter describes the software and hardware requirements, installation procedure, project structure, and execution steps required to successfully deploy and run the N100 Financial Intelligence Dashboard. The application is built using Python and consists of multiple integrated modules including the ETL pipeline, SQLite database, FastAPI backend, and Streamlit dashboard. Following the installation steps correctly ensures that all components function seamlessly and provide accurate financial analytics.

3.2 System Requirements

Before installing the application, the following minimum system requirements should be satisfied.

Hardware Requirements

Component	Requirement
Processor	Intel Core i5 or equivalent
RAM	Minimum 8 GB (16 GB Recommended)
Storage	5 GB Free Disk Space
Internet	Required for package installation

Software Requirements

Software	Version
Operating System	Windows 10/11, macOS, or Linux
Python	3.11 or above
Visual Studio Code	Latest Version
Git	Latest Version
SQLite	Latest Version

Software	Version
Streamlit	Latest Version
FastAPI	Latest Version

3.3 Project Directory Structure

The project follows a modular folder structure to separate data processing, backend services, dashboard components, reports, and testing modules.

N100_Financial_Intelligence/

```

|
├── config/
├── data/
│   ├── raw/
│   └── processed/
├── db/
│   └── nifty100.db
├── docs/
├── output/
├── reports/
├── src/
│   ├── analytics/
│   ├── api/
│   ├── dashboard/
│   ├── etl/
│   ├── nlp/
│   ├── reports/
│   └── screener/
├── tests/
├── requirements.txt
└── README.md

```

The modular organization improves maintainability by separating each functional component into independent directories.

3.4 Installing Dependencies

Clone or download the project into the local system and navigate to the project directory.

Create a virtual environment to isolate project dependencies.

```
python -m venv .venv
```

Activate the virtual environment.

Windows

```
.venv\Scripts\activate
```

macOS/Linux

```
source .venv/bin/activate
```

Install all required packages using the requirements file.

```
pip install -r requirements.txt
```

The installation process automatically installs all required Python libraries including Streamlit, FastAPI, Pandas, Plotly, NumPy, ReportLab, and Pytest.

3.5 Database Setup

The application uses SQLite as its primary database management system.

Execute the ETL pipeline to process the raw financial datasets and populate the database.

```
python src/etl/create_database.py
```

This process performs the following operations:

- Reads raw financial datasets.
- Cleans and validates the data.
- Standardizes company information.
- Creates SQLite database tables.
- Loads processed records into the database.

After successful execution, the database file **nifty100.db** is created inside the **db** directory.

3.6 Running the FastAPI Backend

The FastAPI server provides RESTful APIs used by the dashboard and external applications.

Start the server using the following command:

```
uvicorn src.api.main:app --reload
```

After successful execution, the API server becomes available at:

http://localhost:8000

Interactive API documentation can be accessed using Swagger UI.

http://localhost:8000/docs

Swagger automatically lists all available endpoints, request parameters, and response formats.

3.7 Running the Streamlit Dashboard

The dashboard is developed using Streamlit and provides the graphical interface of the application.

Launch the dashboard using:

```
streamlit run dashboard/Home.py
```

Once executed, Streamlit automatically opens the dashboard in the default web browser.

The homepage displays the Industry Overview dashboard from which users can navigate to the remaining analytical modules.

3.8 Verifying Successful Installation

The installation is considered successful if the following conditions are satisfied:

- The SQLite database is successfully created.
- The FastAPI server starts without errors.
- Swagger documentation opens correctly.
- The Streamlit dashboard loads successfully.
- Company information is displayed correctly.
- Dashboard filters respond properly.
- Charts and KPIs are visible.
- Reports can be generated and downloaded.

3.9 Common Installation Issues

Some common installation problems and their solutions are shown below.

Problem	Possible Cause	Solution
ModuleNotFound Error	Missing package	Install dependencies using <code>pip install -r requirements.txt</code>
Database not found	ETL not executed	Run the database creation script

Problem	Possible Cause	Solution
API not starting	Incorrect Python environment	Activate the virtual environment
Dashboard not loading	Streamlit not installed	Install Streamlit using pip
Empty dashboard	Missing database records	Verify that the ETL process completed successfully

3.10 Chapter Summary

This chapter described the installation and configuration process for the N100 Financial Intelligence Dashboard. It covered the hardware and software requirements, project directory structure, dependency installation, database setup, backend deployment, dashboard execution, and troubleshooting steps. Following these procedures ensures that the application is installed correctly and ready for financial analysis. The next chapter provides a detailed walkthrough of each dashboard module and explains how users can interact with the analytical features of the system.

CHAPTER 4: DASHBOARD MODULES

4.1 Introduction

The dashboard is the primary interface of the N100 Financial Intelligence Dashboard, providing users with interactive tools to analyze financial performance, compare companies, evaluate sectors, and generate analytical reports. Built using Streamlit and Plotly, it enables users to explore financial information through dynamic charts, KPI cards, filters, and tables.

Each dashboard module focuses on a specific aspect of financial analysis while maintaining seamless integration with the underlying SQLite database and FastAPI backend. The modules are designed to assist analysts, investors, researchers, and students in making data-driven financial decisions.

4.2 Dashboard Navigation

The dashboard consists of multiple interconnected pages that can be accessed through the navigation sidebar. Each page serves a distinct analytical purpose and allows users to filter data based on company, sector, or financial metrics.

The major dashboard modules include:

- Industry Overview
- Company Profile
- Stock Screener
- Sector Analysis
- Peer Comparison
- Valuation Analysis
- Market Capitalization
- Portfolio Analysis
- AI Insights
- Reports and Downloads

The modular design ensures ease of navigation and enhances the overall user experience.

4.3 Industry Overview Dashboard

The Industry Overview dashboard provides a high-level summary of the financial performance of companies included in the Nifty 100 index. It enables users to quickly assess market trends and overall industry health.

Features

- Total Market Capitalization
- Number of Companies

- Sector Distribution
- Average Financial Ratios
- Industry-wise Performance
- Top Performing Companies
- Bottom Performing Companies

Visualizations

- KPI Cards
- Bar Charts
- Pie Charts
- Line Graphs
- Sector Distribution Charts

The dashboard provides an overview before users proceed to detailed company-level analysis.

4.4 Company Profile Dashboard

The Company Profile module presents comprehensive financial information for an individual company.

Users can select a company from the dropdown menu to access detailed financial data.

Information Displayed

- Company Name
- Sector
- Industry
- Market Capitalization
- Revenue
- Net Profit
- Earnings Per Share (EPS)
- Book Value
- Dividend Yield
- Current Share Price
- Price-to-Earnings Ratio (P/E)

The module also displays historical financial performance using interactive charts.

Benefits

- Company-specific financial analysis
- Historical trend visualization
- Quick access to financial indicators

4.5 Stock Screener Dashboard

The Stock Screener enables users to identify companies based on customizable financial criteria.

Users can apply filters such as:

- Market Capitalization
- Revenue Growth
- Return on Equity (ROE)
- Return on Capital Employed (ROCE)
- Debt-to-Equity Ratio
- Price-to-Earnings Ratio
- Dividend Yield
- Net Profit Margin

The dashboard instantly updates the list of companies satisfying the selected criteria.

Advantages

- Saves manual screening effort.
- Identifies investment opportunities.
- Supports customized financial analysis.

4.6 Sector Analysis Dashboard

The Sector Analysis dashboard compares financial performance across different industries.

Users can select any sector to analyze collective financial metrics.

Metrics Available

- Average Revenue
- Average Profit
- Market Capitalization
- Profit Margin
- Revenue Growth
- ROE
- ROCE
- EPS

Charts Used

- Bar Charts
- Line Charts
- Pie Charts
- Heatmaps

This module helps identify high-performing industries and compare sector-wise financial health.

4.7 Peer Comparison Dashboard

The Peer Comparison module enables users to compare multiple companies simultaneously.

Companies belonging to the same industry can be evaluated using various financial parameters.

Comparison Metrics

- Revenue
- Net Profit
- ROE
- ROCE
- EPS
- Debt-to-Equity Ratio
- Current Ratio
- Market Capitalization

Visualization Methods

- Radar Charts
- Comparative Bar Charts
- Financial Tables

This feature assists analysts in identifying the strongest companies within a sector.

4.8 Valuation Analysis Dashboard

The Valuation Analysis dashboard evaluates whether a company's stock appears undervalued or overvalued based on financial indicators.

Metrics Included

- Price-to-Earnings Ratio (P/E)
- Price-to-Book Ratio (P/B)
- Enterprise Value
- EV/EBITDA
- Earnings Per Share
- Book Value

The module provides graphical comparisons between companies and industry averages.

Benefits

- Supports investment decision-making.
- Highlights valuation differences.

- Simplifies company comparison.

4.9 Market Capitalization Dashboard

This module categorizes companies according to their market capitalization and provides insights into their relative size and contribution to the index.

Features

- Market Cap Distribution
- Largest Companies
- Sector-wise Market Cap
- Company Rankings

Charts

- Pie Charts
- Bar Charts
- Treemaps

Users can understand the contribution of individual companies and sectors to the overall market.

4.10 Portfolio Analysis Dashboard

The Portfolio Analysis module allows users to evaluate a selected portfolio of companies.

Key Features

- Portfolio Allocation
- Company Weightage
- Total Investment Value
- Expected Returns
- Risk Distribution
- Sector Diversification

Visualizations

- Allocation Pie Charts
- Portfolio Tables
- Performance Graphs

The module assists investors in monitoring portfolio composition and diversification.

4.11 AI Insights Dashboard

The AI Insights module generates concise summaries based on the financial performance of companies.

Using processed financial data, the system provides observations such as:

- Financial strengths
- Financial weaknesses
- Growth opportunities
- Investment risks
- Overall business performance

These insights reduce the time required for manual financial interpretation and improve analytical efficiency.

4.12 Reports and Downloads

The application enables users to export analytical results for further use.

Supported download formats include:

- PDF Reports
- CSV Files

Generated reports contain:

- Company Information
- Financial Ratios
- Graphs
- Performance Trends
- AI-generated Insights
- Valuation Metrics

This functionality facilitates report sharing and offline analysis.

4.13 Interactive Features

To improve usability, the dashboard incorporates several interactive components:

- Dropdown filters
- Search functionality
- Dynamic charts
- KPI cards
- Sortable data tables
- Download buttons
- Responsive layouts

These features allow users to customize their analysis without requiring technical expertise.

4.14 Advantages of the Dashboard

The dashboard provides several advantages over conventional spreadsheet-based analysis:

- Centralized financial information
- Interactive visualizations
- Automated financial calculations
- Quick company comparisons
- Efficient portfolio analysis
- AI-assisted financial summaries
- Professional report generation
- User-friendly interface

4.15 Chapter Summary

This chapter presented the various dashboard modules available in the N100 Financial Intelligence Dashboard. Each module was explained in terms of its purpose, functionality, key metrics, and visualizations. Together, these modules provide a comprehensive platform for analyzing company performance, comparing industries, evaluating investment opportunities, and generating analytical reports. The next chapter describes the RESTful API services that power the dashboard and enable communication between the frontend and the backend.

CHAPTER 5: API IMPLEMENTATION, TESTING AND REPORT GENERATION

5.1 Introduction

The N100 Financial Intelligence Dashboard is built on a robust backend architecture that enables seamless communication between the database and the user interface. The backend is developed using **FastAPI**, which exposes RESTful APIs for retrieving financial information from the SQLite database. These APIs serve as the foundation for the Streamlit dashboard by providing real-time access to company profiles, financial ratios, sector analysis, valuation metrics, and portfolio information.

In addition to API implementation, the project includes comprehensive testing procedures to ensure reliability, accuracy, and performance. Automated report generation further enhances the usability of the application by allowing users to export financial analyses in PDF and CSV formats.

5.2 FastAPI Backend Implementation

The backend of the application is developed using **FastAPI**, a modern Python web framework known for its high performance and automatic API documentation. The backend acts as an intermediary between the SQLite database and the Streamlit frontend.

The major responsibilities of the backend include:

- Processing user requests.
- Executing SQL queries on the SQLite database.
- Returning structured JSON responses.
- Handling errors and invalid requests.
- Supporting report generation.
- Providing data for dashboard visualizations.

The modular architecture ensures that each API performs a dedicated task, making the system easier to maintain and extend.

5.3 REST API Endpoints

The dashboard communicates with the backend through multiple RESTful API endpoints. Each endpoint retrieves specific financial information required by different dashboard modules.

The primary API endpoints implemented in the project are shown below.

API Endpoint	Description
/health	Verifies whether the server is running successfully

API Endpoint	Description
/companies	Retrieves company profile information
/screener	Returns companies matching selected financial filters
/sectors	Provides sector-wise financial analysis
/peers	Compares companies within the same industry
/valuation	Displays valuation metrics for selected companies
/market-cap	Returns market capitalization statistics
/portfolio	Performs portfolio analysis and allocation
/documents	Retrieves downloadable reports and documents

Each endpoint accepts user requests, processes the necessary database queries, and returns structured JSON responses that are displayed on the dashboard.

5.4 API Documentation

One of the key advantages of using FastAPI is the automatic generation of interactive API documentation through **Swagger UI**.

The documentation includes:

- List of available endpoints.
- Request parameters.
- Response formats.
- HTTP methods.
- Status codes.
- Interactive API testing.

Developers can test each endpoint directly through the browser without requiring additional software, simplifying debugging and development.

5.5 Testing Methodology

To ensure that the application performs reliably under different scenarios, multiple testing techniques were employed throughout the development process.

The testing process focused on validating the correctness of calculations, API functionality, dashboard responsiveness, database connectivity, and report generation.

The project primarily utilizes **Pytest** for automated testing.

The following testing approaches were adopted:

Unit Testing

Unit testing verifies individual modules independently.

Examples include:

- Financial ratio calculations.
- Database utility functions.
- ETL processing functions.
- API helper methods.

Integration Testing

Integration testing verifies communication between different components of the application.

Examples include:

- FastAPI and SQLite integration.
- Dashboard and API communication.
- ETL pipeline and database loading.
- Report generation using processed data.

Functional Testing

Functional testing ensures that every feature behaves according to the project requirements.

Examples include:

- Company search functionality.
- Dashboard filters.
- Peer comparison.
- Sector analysis.
- Portfolio analysis.
- Report downloads.

API Testing

API testing verifies the correctness of backend endpoints.

The following aspects were tested:

- Valid requests.

- Invalid requests.
- Missing parameters.
- Database connectivity.
- Response format.
- HTTP status codes.

User Interface Testing

The Streamlit dashboard was tested for:

- Navigation.
- Responsiveness.
- Interactive filters.
- Graph rendering.
- Table visualization.
- Download functionality.

5.6 Test Results

The application successfully passed all major functional tests.

The testing confirmed:

- Correct database connectivity.
- Accurate financial calculations.
- Successful API communication.
- Proper dashboard rendering.
- Correct report generation.
- Stable application performance.

Automated testing using Pytest ensured that future updates can be verified efficiently without affecting existing functionality.

5.7 Performance Evaluation

Performance testing was conducted to evaluate the responsiveness and efficiency of the application.

The following parameters were evaluated:

- Dashboard loading time.
- Database query execution.
- API response time.
- Memory utilization.

- Report generation speed.

Database indexing and optimized SQL queries significantly improved application performance by reducing query execution time and ensuring smooth dashboard interactions.

5.8 Report Generation

The dashboard provides users with the ability to export analytical reports for offline use.

Supported formats include:

- PDF Reports
- CSV Files

The generated reports typically contain:

- Company Overview
- Financial Statements
- Ratio Analysis
- Performance Charts
- Sector Analysis
- Valuation Metrics
- AI-generated Insights
- Portfolio Summary

These reports assist analysts and investors in documenting and sharing financial analyses.

5.9 Error Handling and Troubleshooting

To improve system reliability, appropriate error-handling mechanisms have been incorporated throughout the application.

Common errors include:

Error	Possible Cause	Solution
Database Connection Failed	Database not initialized	Execute ETL pipeline and recreate the database
Module Not Found	Missing Python package	Install dependencies using <code>pip install -r requirements.txt</code>
API Not Responding	FastAPI server stopped	Restart the backend server

Error	Possible Cause	Solution
Dashboard Not Loading	Streamlit execution issue	Restart the Streamlit application
Empty Dashboard	Missing financial records	Verify successful data loading into the database

Meaningful error messages guide users toward resolving issues efficiently while preventing unexpected application crashes.

5.10 Security and Reliability

Although the application is primarily intended for analytical purposes, several measures have been implemented to improve system reliability and maintainability.

These include:

- Input validation before processing requests.
- Exception handling to prevent application failures.
- Modular API architecture for easier maintenance.
- Read-only analytical operations to protect stored financial data.
- Organized project structure supporting scalability and future enhancements.

5.11 Chapter Summary

This chapter discussed the implementation of the FastAPI backend, RESTful API endpoints, automated testing methodology, performance evaluation, report generation process, and troubleshooting mechanisms. The backend provides reliable communication between the SQLite database and the Streamlit dashboard, while comprehensive testing ensures application stability and accuracy. The report generation feature further enhances usability by enabling users to export professional financial analyses for offline reference.

CHAPTER 6: PROJECT IMPLEMENTATION AND CONCLUSION

6.1 Introduction

This chapter summarizes the successful implementation of the N100 Financial Intelligence Dashboard. It highlights the completed deliverables, implemented modules, testing outcomes, and the overall achievements of the project. The project was developed according to the planned objectives and successfully integrates data processing, backend services, database management, and interactive dashboards into a single financial analytics platform.

6.2 Project Implementation

The project was successfully implemented by integrating all major components into a unified system. The development process followed a modular approach, ensuring that each component functioned independently while maintaining seamless communication with the rest of the application.

The implementation included:

- Development of an automated ETL pipeline for processing financial datasets.
- Creation of a centralized SQLite database to store cleaned and structured financial data.
- Implementation of FastAPI-based RESTful APIs for efficient backend communication.
- Design and development of an interactive Streamlit dashboard for financial analysis.
- Integration of multiple financial analysis modules such as company profiles, sector analysis, peer comparison, valuation analysis, and portfolio analysis.
- Implementation of AI-generated financial insights to provide concise summaries of company performance.
- Development of report generation functionality supporting PDF and CSV exports.

All modules were integrated successfully and operated as intended throughout the testing phase.

6.3 Dashboard Implementation

The dashboard serves as the primary interface of the application and was successfully developed using Streamlit.

The implemented dashboard includes the following analytical modules:

- Industry Overview Dashboard
- Company Profile Dashboard
- Stock Screener
- Sector Analysis
- Peer Comparison

- Valuation Analysis
- Market Capitalization Analysis
- Portfolio Analysis
- AI Insights
- Report Generation

Each dashboard page provides interactive filters, KPI cards, charts, and tables, enabling users to analyze financial information efficiently. The dashboard successfully retrieves real-time information from the backend APIs and displays it through dynamic visualizations.

6.4 Project Deliverables

The project achieved all planned deliverables successfully.

The final deliverables include:

- Fully functional ETL pipeline.
- Cleaned and processed financial datasets.
- SQLite database containing structured financial information.
- FastAPI backend with RESTful API endpoints.
- Interactive Streamlit dashboard.
- Financial ratio analysis module.
- Company comparison and sector analysis modules.
- Portfolio analysis functionality.
- AI-generated financial insights.
- PDF and CSV report generation.
- API documentation using Swagger.
- Comprehensive testing using Pytest.
- Technical documentation and Analyst Guide.

These deliverables collectively satisfy the functional requirements defined at the beginning of the project.

6.5 Testing and Validation

The completed system was tested to verify the correctness of all implemented features.

The testing process confirmed that:

- The ETL pipeline successfully processed and loaded financial data.
- Database operations executed correctly.
- RESTful APIs returned accurate responses.

- Dashboard pages displayed correct financial information.
- Interactive filters and charts functioned properly.
- Report generation worked successfully.
- The application remained stable during execution.

The successful completion of testing verified that the implemented system satisfies the intended functional requirements.

6.6 Project Outcomes

The completed project provides a centralized platform for financial analysis by combining automated data processing, backend services, and interactive visualization.

The implemented system enables users to:

- Analyze financial performance of Nifty 100 companies.
- Compare companies across multiple financial metrics.
- Evaluate sector-wise performance.
- Screen companies using customized financial filters.
- Analyze portfolio composition.
- Generate downloadable analytical reports.
- Obtain AI-assisted financial summaries.

The project successfully reduces manual effort, improves analytical efficiency, and presents financial information in an intuitive and user-friendly manner.

6.7 Conclusion

The **N100 Financial Intelligence Dashboard** was successfully designed, developed, tested, and implemented according to the project objectives. All planned modules and deliverables were completed, including the ETL pipeline, SQLite database, FastAPI backend, Streamlit dashboard, financial analysis modules, AI insights, and report generation features.

The completed application provides a reliable and scalable financial analytics platform capable of transforming raw financial data into meaningful insights. By integrating automated data processing with interactive visualization, the project simplifies financial analysis and supports informed decision-making for analysts, investors, researchers, and students.

The successful completion of this project demonstrates the effective application of Python, FastAPI, SQLite, and Streamlit in developing a modern financial intelligence solution.

